

# GSpot Gaming Hub

Entity Relationship Diagram — Entity Descriptions

Generated: | Total Entities: 12

## GSpot Gaming Hub — ERD Entity Descriptions

### ENTITY 1 — users

**Description:** The central entity of the entire system. Stores all system accounts regardless of role — customers who make reservations, shopkeepers who manage sessions, and owners who access reports. Every other entity in the system either belongs to or is created by a user record. **Attributes:**

| Attribute                 | Type                              | Description                                      |
|---------------------------|-----------------------------------|--------------------------------------------------|
| user_id                   | INT, PK, AUTO_INCREMENT           | Unique identifier for every user                 |
| email                     | VARCHAR(100), UNIQUE              | Login email address                              |
| password_hash             | VARCHAR(255)                      | Bcrypt-hashed password                           |
| full_name                 | VARCHAR(100)                      | Display name                                     |
| phone                     | VARCHAR(20), NULLABLE             | Contact number                                   |
| role                      | ENUM(customer, shopkeeper, owner) | Determines access level                          |
| status                    | ENUM(active, inactive)            | Account active/deactivated                       |
| consecutive_cancellations | TINYINT UNSIGNED                  | Count of non-grace cancellations (3-strike rule) |
| reservation_banned_until  | DATETIME, NULLABLE                | Booking ban expiry; NULL = not banned            |
| email_verified            | TINYINT(1)                        | 1 = email confirmed, 0 = pending                 |
| created_at                | DATETIME                          | Account creation timestamp                       |

**Primary Key:** user\_id **Foreign Keys:** None (top-level entity) **Relationships:**

| Related Entity            | Type      | Description                                |
|---------------------------|-----------|--------------------------------------------|
| notifications             | 1-to-Many | One user receives many notifications       |
| reservations              | 1-to-Many | One user makes many reservations           |
| gaming_sessions           | 1-to-Many | One user has many gaming sessions          |
| reservation_cancellations | 1-to-Many | One user has many cancellation log entries |
| reschedules               | 1-to-Many | One user (admin) performs many reschedules |
| transactions              | 1-to-Many | One user has many payment transactions     |
| tournament_participants   | 1-to-Many | One user joins many tournaments            |

**Cardinality:** - 1 user → 0..N reservations - 1 user → 0..N gaming\_sessions - 1 user → 0..N transactions - 1 user → 0..N tournament\_participants - 1 user → 0..N notifications **Rules & Constraints:** - Email must be unique across the system - consecutive\_cancellations increments on each non-grace-period cancellation - When consecutive\_cancellations reaches 3, reservation\_banned\_until is set to NOW() + ban\_duration ; the user cannot create new reservations until this timestamp

passes - Streak resets to 0 after any successful completed gaming session - `role` determines system access: customers see the customer dashboard; shopkeepers and owners access the admin panel - Accounts with `status = inactive` cannot log in

---

## ENTITY 2 — consoles

**Description:** Represents the physical gaming units available in the shop — PS5 and Xbox Series X consoles. Each unit has its own record so availability can be tracked per unit. Staff assigns a specific console to a session or reservation. **Attributes:**

| Attribute                 | Type                                 | Description                      |
|---------------------------|--------------------------------------|----------------------------------|
| <code>console_id</code>   | INT, PK, AUTO_INCREMENT              | Unique unit identifier           |
| <code>console_name</code> | VARCHAR(50)                          | Human label (e.g., "PS5 Unit 1") |
| <code>console_type</code> | ENUM(PS5, Xbox Series X)             | Hardware type                    |
| <code>unit_number</code>  | VARCHAR(10), UNIQUE                  | Short code (e.g., "PS5-01")      |
| <code>status</code>       | ENUM(available, in_use, maintenance) | Real-time availability           |
| <code>hourly_rate</code>  | DECIMAL(10,2)                        | Rate charged per hour            |
| <code>created_at</code>   | DATETIME                             | When the unit was registered     |

**Primary Key:** `console_id` **Foreign Keys:** None **Relationships:**

| Related Entity  | Type      | Description                                  |
|-----------------|-----------|----------------------------------------------|
| reservations    | 1-to-Many | One console can have many reservations       |
| gaming_sessions | 1-to-Many | One console can host many sessions over time |

**Cardinality:** - 1 console → 0..N reservations (across different dates) - 1 console → 0..N gaming\_sessions (across time) **Rules &**

**Constraints:** - Only one active session can run per console at any time ( `status = in_use` ) - `unit_number` must be unique — no two units share the same code - Units with `status = maintenance` cannot be assigned to reservations or sessions - Deletion is restricted if active sessions or future reservations reference the unit

### ENTITY 3 — games

**Description:** The game library — each record represents a title available to play on a specific console type. Used to associate tournaments with a specific game. Customers can also submit game requests referencing titles not yet in this library. **Attributes:**

| Attribute      | Type                           | Description                          |
|----------------|--------------------------------|--------------------------------------|
| game_id        | INT, PK, AUTO_INCREMENT        | Unique game identifier               |
| game_name      | VARCHAR(150)                   | Title of the game                    |
| console_type   | ENUM(PSS, Xbox Series X, Both) | Supported platform(s)                |
| genre          | VARCHAR(50), NULLABLE          | Game genre                           |
| is_available   | TINYINT(1)                     | 1 = currently installed and playable |
| is_new_release | TINYINT(1)                     | 1 = flagged as new                   |
| added_date     | DATE                           | When added to the library            |

**Primary Key:** game\_id **Foreign Keys:** None **Relationships:**

| Related Entity | Type      | Description                                  |
|----------------|-----------|----------------------------------------------|
| tournaments    | 1-to-Many | One game can be featured in many tournaments |

**Cardinality:** - 1 game → 0..N tournaments **Rules & Constraints:** - A game cannot be deleted if it is referenced by an active or upcoming tournament - is\_available = 0 hides the game from the public game list but does not affect existing tournament records

**ENTITY 4 — notifications**

**Description:** Stores all system-generated messages sent to a specific user — booking confirmations, cancellation results, ban notices, reschedule alerts, and tournament updates. Displayed in the user's notification panel. **Attributes:**

| Attribute                    | Type                    | Description                                                                                      |
|------------------------------|-------------------------|--------------------------------------------------------------------------------------------------|
| <code>notification_id</code> | INT, PK, AUTO_INCREMENT | Unique notification identifier                                                                   |
| <code>user_id</code>         | INT, FK → users.user_id | Recipient of the notification                                                                    |
| <code>type</code>            | ENUM / VARCHAR          | Category: reservation_confirmed, cancel_notice, ban_notice, reschedule_notice, tournament_update |
| <code>message</code>         | TEXT                    | Full notification text                                                                           |
| <code>is_read</code>         | TINYINT(1)              | 0 = unread, 1 = read                                                                             |
| <code>created_at</code>      | DATETIME                | When the notification was generated                                                              |

**Primary Key:** `notification_id` **Foreign Keys:** `user_id` → `users.user_id` **Relationships:**

| Related Entity | Type      | Description                           |
|----------------|-----------|---------------------------------------|
| users          | Many-to-1 | Many notifications belong to one user |

**Cardinality:** - 1 user → 0..N notifications **Rules & Constraints:** - Notifications are user-scoped — a user only sees their own - System generates notifications automatically on: reservation confirmation, cancellation, ban, reschedule, tournament slot confirmed - `is_read` is set to 1 when the user opens or dismisses the notification

## ENTITY 5 — reservations

**Description:** The core booking entity. When a customer books a gaming session in advance, a reservation record is created. It tracks which console type was booked, the date and time, the rental mode, downpayment details, and the lifecycle status. Staff converts confirmed reservations into live sessions. **Attributes:**

| Attribute                       | Type                                                                                                                | Description                                           |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| <code>reservation_id</code>     | INT, PK, AUTO_INCREMENT                                                                                             | Unique reservation identifier                         |
| <code>user_id</code>            | INT, FK → users.user_id                                                                                             | Customer who made the booking                         |
| <code>console_id</code>         | INT, FK → consoles.console_id, NULLABLE                                                                             | Specific unit assigned (nullable until staff assigns) |
| <code>created_by</code>         | INT, FK → users.user_id                                                                                             | Who submitted (customer = self, staff = admin ID)     |
| <code>console_type</code>       | ENUM(PS5, Xbox Series X)                                                                                            | Console type requested                                |
| <code>rental_mode</code>        | ENUM(hourly, open_time, unlimited)                                                                                  | Billing mode                                          |
| <code>planned_minutes</code>    | INT, NULLABLE                                                                                                       | Duration in minutes (hourly mode only)                |
| <code>reserved_date</code>      | DATE                                                                                                                | Scheduled date                                        |
| <code>reserved_time</code>      | TIME                                                                                                                | Scheduled start time                                  |
| <code>downpayment_amount</code> | DECIMAL(10,2)                                                                                                       | Amount paid as downpayment                            |
| <code>downpayment_method</code> | ENUM(cash, gcash, credit_card), NULLABLE                                                                            | How the downpayment was made                          |
| <code>status</code>             | ENUM(pending, confirmed, converted, cancelled, no_show)                                                             | Lifecycle status                                      |
| <code>cancel_reason_type</code> | ENUM(schedule_change, found_alternative, budget_issue, technical_issue, emergency, other, admin_decision), NULLABLE | Category of cancellation reason                       |
| <code>cancelled_by</code>       | ENUM(user, admin), NULLABLE                                                                                         | Who initiated the cancellation                        |
| <code>created_at</code>         | DATETIME                                                                                                            | When the reservation was submitted                    |
| <code>updated_at</code>         | DATETIME                                                                                                            | Last modification timestamp                           |

**Primary Key:** `reservation_id` **Foreign Keys:** - `user_id` → `users.user_id` - `console_id` → `consoles.console_id` (SET NULL on delete) - `created_by` → `users.user_id` **Relationships:**

| Related Entity            | Type      | Description                                       |
|---------------------------|-----------|---------------------------------------------------|
| users                     | Many-to-1 | Many reservations belong to one customer          |
| consoles                  | Many-to-1 | Many reservations may be assigned to one console  |
| reservation_cancellations | 1-to-1    | One reservation has at most one cancellation log  |
| reschedules               | 1-to-1    | One reservation may have one reschedule           |
| gaming_sessions           | 1-to-1    | One confirmed reservation converts to one session |

**Cardinality:** - Many reservations → 1 user - Many reservations → 1 console (assigned) - 1 reservation → 0..1 cancellation record - 1 reservation → 0..1 reschedule record **Rules & Constraints:** - Booking date cannot be more than 1 month (31 days) from today - Only one active reservation per customer at a time is enforced at the application level - Availability is checked using duration-based overlap ( `reserved_time` + `planned_minutes` ), not a flat ±2-hour window - **30-minute grace period:** If cancelled within 30 minutes of `created_at`, the cancellation is penalty-free ( `consecutive_cancellations` is not incremented) - If cancelled after 30 minutes, the cancellation is non-grace and the user's streak increments - `cancel_reason_type` and `cancelled_by` are required at the time of cancellation — blank reasons are rejected - **No-Refund Policy:** Downpayment is never refunded regardless of cancellation reason - Status `converted` means the reservation has become a live gaming session

## ENTITY 6 — reservation\_cancellations

**Description:** A dedicated audit log table. Every time a reservation is cancelled — whether by the customer or by an admin — a detailed record is written here. This is separate from the status field in `reservations` to preserve full history for reporting and the 3-strike enforcement logic. **Attributes:**

| Attribute                            | Type                                  | Description                                 |
|--------------------------------------|---------------------------------------|---------------------------------------------|
| <code>cancel_id</code>               | INT, PK, AUTO_INCREMENT               | Unique cancellation log ID                  |
| <code>reservation_id</code>          | INT, FK → reservations.reservation_id | The cancelled reservation                   |
| <code>user_id</code>                 | INT, FK → users.user_id               | The customer who owned the reservation      |
| <code>cancelled_by</code>            | ENUM(user, admin)                     | Who initiated the cancellation              |
| <code>cancel_reason_type</code>      | ENUM(...)                             | Category of cancellation reason             |
| <code>cancel_reason_detail</code>    | TEXT, NULLABLE                        | Free-text elaboration of reason             |
| <code>is_within_grace (30min)</code> | TINYINT(1)                            | 1 = cancelled within 30-minute grace window |
| <code>reserved_date</code>           | DATE                                  | The original booking date (snapshot)        |
| <code>downpayment_amount</code>      | DECIMAL(10,2)                         | Downpayment at time of cancellation         |
| <code>refund_issued</code>           | TINYINT(1)                            | Always 0 — No-Refund Policy                 |
| <code>cancelled_at</code>            | DATETIME                              | Exact timestamp of cancellation             |

**Primary Key:** `cancel_id` **Foreign Keys:** - `reservation_id` → `reservations.reservation_id` (CASCADE DELETE) - `user_id` → `users.user_id` **Relationships:**

| Related Entity | Type      | Description                                     |
|----------------|-----------|-------------------------------------------------|
| reservations   | Many-to-1 | One cancellation log belongs to one reservation |
| users          | Many-to-1 | One cancellation log records one customer       |

**Cardinality:** - 1 reservation → 0..1 cancellation record - 1 user → 0..N cancellation records (one per cancellation event) **Rules & Constraints:** - `cancelled_by = admin` entries do NOT trigger streak increment — only `cancelled_by = user` outside the grace window does - `refund_issued` is always 0 — enforcing the No-Refund Policy at the data level - `is_within_grace` determines whether the cancellation counts against the 3-strike rule - The record is the source of truth for the "My Cancellations" history view - Reason detail cannot be blank — enforced at the application layer before write

## ENTITY 7 — reschedules

**Description:** Stores every administrative rescheduling event. When an admin reschedules a reservation due to external reasons (typhoon, power outage, emergency), the original date/time is preserved and the new schedule is recorded. Rescheduling never affects the customer's cancellation streak. **Attributes:**

| Attribute                   | Type                                          | Description                        |
|-----------------------------|-----------------------------------------------|------------------------------------|
| <code>reschedule_id</code>  | INT, PK, AUTO_INCREMENT                       | Unique reschedule identifier       |
| <code>reservation_id</code> | INT, FK → reservations.reservation_id         | The affected reservation           |
| <code>rescheduled_by</code> | INT, FK → users.user_id                       | Admin who performed the reschedule |
| <code>original_date</code>  | DATE                                          | The original reserved date         |
| <code>original_time</code>  | TIME                                          | The original reserved time         |
| <code>new_date</code>       | DATE                                          | The rescheduled date               |
| <code>new_time</code>       | TIME                                          | The rescheduled time               |
| <code>reason_type</code>    | ENUM(typhoon, power_outage, emergency, other) | Category of reason                 |
| <code>reason_detail</code>  | TEXT, NULLABLE                                | Free-text elaboration              |
| <code>created_at</code>     | DATETIME                                      | When the reschedule was recorded   |

**Primary Key:** `reschedule_id` **Foreign Keys:** - `reservation_id` → `reservations.reservation_id` - `rescheduled_by` → `users.user_id` **Relationships:**

| Related Entity | Type      | Description                                      |
|----------------|-----------|--------------------------------------------------|
| reservations   | Many-to-1 | One reschedule record belongs to one reservation |
| users          | Many-to-1 | One admin (user) performs many reschedules       |

**Cardinality:** - 1 reservation → 0..1 reschedule record (one active reschedule per reservation) - 1 admin user → 0..N reschedule records **Rules & Constraints:** - Rescheduling does NOT increment `consecutive_cancellations` — it is not treated as a cancellation - Only staff/admin ( `role = shopkeeper` or `owner` ) can create reschedule records - `reason_type` must be selected from the defined ENUM values - A notification is automatically sent to the customer when a reschedule is recorded - The original reservation record is updated with the new date/time; this table preserves the full audit trail

## ENTITY 8 — gaming\_sessions

**Description:** Tracks every live and completed gaming rental session. Created when staff starts a session — either by converting a confirmed reservation or by registering a walk-in customer. Records the full billing lifecycle including start time, end time, rental mode, cost, and any extensions. **Attributes:**

| Attribute        | Type                               | Description                                   |
|------------------|------------------------------------|-----------------------------------------------|
| session_id       | INT, PK, AUTO_INCREMENT            | Unique session identifier                     |
| user_id          | INT, FK → users.user_id            | Customer playing (or walk-in user ID)         |
| console_id       | INT, FK → consoles.console_id      | The assigned console unit                     |
| created_by       | INT, FK → users.user_id            | Staff who started the session                 |
| rental_mode      | ENUM(hourly, open_time, unlimited) | Billing mode                                  |
| start_time       | DATETIME                           | When the session began                        |
| end_time         | DATETIME, NULLABLE                 | When the session ended (NULL if still active) |
| duration_minutes | INT, NULLABLE                      | Actual elapsed minutes at end                 |
| extended_minutes | INT DEFAULT 0                      | Total minutes added via approved extensions   |
| total_cost       | DECIMAL(10,2), NULLABLE            | Final billed amount                           |
| status           | ENUM(active, completed, cancelled) | Session lifecycle status                      |

**Primary Key:** session\_id **Foreign Keys:** - user\_id → users.user\_id - console\_id → consoles.console\_id - created\_by → users.user\_id **Relationships:**

| Related Entity     | Type      | Description                                    |
|--------------------|-----------|------------------------------------------------|
| users              | Many-to-1 | Many sessions belong to one customer           |
| consoles           | Many-to-1 | Many sessions run on one console (over time)   |
| session_extensions | 1-to-Many | One session can have many extension requests   |
| transactions       | 1-to-Many | One session generates one or more transactions |

**Cardinality:** - 1 user → 0..N sessions - 1 console → 0..N sessions (serially, not simultaneously) - 1 session → 0..N extensions - 1 session → 1..N transactions **Rules & Constraints:** - **Only one active session per console at a time** — validated before creating a new session - **Starter rate:** ₱50 for the first 30 minutes; ₱80/hr thereafter - **Unlimited mode:** flat fee regardless of duration - **Bonus time:** 30 free minutes awarded per every 2 hours of paid play - extended\_minutes accumulates across all approved extension requests - Staff must end the session ( end\_time set) before a transaction can be finalized - After end\_time is set, admin may retroactively correct it — triggering a full cost recalculation - Walk-in sessions may use a generic/guest user ID if the customer has no registered account

## ENTITY 9 — session\_extensions

**Description:** Tracks every request to extend an active gaming session's duration. Customers can request extensions; staff can also add them directly. Each extension has a status and records the extra minutes and cost charged. **Attributes:**

| Attribute      | Type                                     | Description                                 |
|----------------|------------------------------------------|---------------------------------------------|
| extension_id   | INT, PK, AUTO_INCREMENT                  | Unique extension identifier                 |
| session_id     | INT, FK → gaming_sessions.session_id     | The session being extended                  |
| requested_by   | INT, FK → users.user_id                  | Who requested the extension                 |
| approved_by    | INT, FK → users.user_id, NULLABLE        | Staff who approved (NULL if pending/denied) |
| extra_minutes  | INT                                      | Minutes to add                              |
| extra_cost     | DECIMAL(10,2)                            | Cost of the extension                       |
| payment_method | ENUM(cash, gcash, credit_card), NULLABLE | Set when payment is collected               |
| status         | ENUM(pending, approved, denied)          | Approval state                              |
| requested_at   | DATETIME                                 | Timestamp of request                        |
| resolved_at    | DATETIME, NULLABLE                       | When staff approved or denied               |

**Primary Key:** extension\_id **Foreign Keys:** - session\_id → gaming\_sessions.session\_id (CASCADE DELETE) - requested\_by → users.user\_id - approved\_by → users.user\_id (NULLABLE) **Relationships:**

| Related Entity  | Type      | Description                                           |
|-----------------|-----------|-------------------------------------------------------|
| gaming_sessions | Many-to-1 | Many extensions belong to one session                 |
| users           | Many-to-1 | Many extensions requested by users; approved by staff |

**Cardinality:** - 1 session → 0..N extension requests - 1 user → 0..N extension requests (as customer or staff) **Rules & Constraints:** - **Extension rate:** ₱40 per 30 minutes - Staff-direct extensions are inserted with status = approved immediately (no pending step) - Customer extension requests start as status = pending and require staff approval - approved\_by is NULL until a staff member acts on the request - On approval, gaming\_sessions.extended\_minutes is incremented by extra\_minutes - **Free bonus time is not charged:** 30 min free per 2 hours paid — this is applied at billing, not at the extension level

## ENTITY 10 — transactions

**Description:** Records every payment event in the system. Each gaming session generates at least one transaction record. Captures the amount, payment method (GCash), processor, and whether the No-Refund Policy was acknowledged. **Attributes:**

| Attribute              | Type                                 | Description                                 |
|------------------------|--------------------------------------|---------------------------------------------|
| transaction_id         | INT, PK, AUTO_INCREMENT              | Unique transaction identifier               |
| session_id             | INT, FK → gaming_sessions.session_id | The session this payment covers             |
| user_id                | INT, FK → users.user_id              | The paying customer                         |
| processed_by           | INT, FK → users.user_id              | Staff who processed the payment             |
| amount                 | DECIMAL(10,2)                        | Amount charged                              |
| payment_method (gcash) | ENUM(cash, gcash, credit_card)       | Payment method used                         |
| payment_status         | ENUM(pending, completed, failed)     | Payment completion state                    |
| no_refund_acknowledged | TINYINT(1)                           | 1 = customer confirmed the no-refund policy |
| transaction_date       | DATETIME                             | When payment was processed                  |

**Primary Key:** transaction\_id **Foreign Keys:** - session\_id → gaming\_sessions.session\_id - user\_id → users.user\_id - processed\_by → users.user\_id **Relationships:**

| Related Entity  | Type      | Description                                                        |
|-----------------|-----------|--------------------------------------------------------------------|
| gaming_sessions | Many-to-1 | Many transactions can belong to one session (initial + extensions) |
| users           | Many-to-1 | Many transactions tied to one customer                             |

**Cardinality:** - 1 session → 1..N transactions - 1 user → 0..N transactions **Rules & Constraints:** - **No-Refund Policy:** Once payment\_status = completed, no refund is issued under any circumstance - GCash (via PayMongo) is the preferred and promoted payment method for online flows - Walk-in sessions use payment\_method = cash - no\_refund\_acknowledged = 1 must be set before a reservation or session payment is finalized - processed\_by references the staff user who collected or verified the payment

## ENTITY 11 — tournaments

**Description:** Represents gaming competitions organized by the shop. Each tournament is tied to a specific game and console type, has a defined schedule and entry fee, and follows a lifecycle from upcoming through to completed. **Attributes:**

| Attribute                     | Type                                                     | Description                     |
|-------------------------------|----------------------------------------------------------|---------------------------------|
| <code>tournament_id</code>    | INT, PK, AUTO_INCREMENT                                  | Unique tournament identifier    |
| <code>game_id</code>          | INT, FK → <code>games.game_id</code>                     | The featured game               |
| <code>tournament_name</code>  | VARCHAR(150)                                             | Name of the tournament          |
| <code>console_type</code>     | ENUM(PS5, Xbox Series X)                                 | Platform for this tournament    |
| <code>start_date</code>       | DATETIME                                                 | Tournament start date and time  |
| <code>end_date</code>         | DATETIME                                                 | Tournament end date and time    |
| <code>entry_fee</code>        | DECIMAL(10,2)                                            | Cost to register                |
| <code>prize_pool</code>       | DECIMAL(10,2)                                            | Total prize amount              |
| <code>max_participants</code> | INT                                                      | Maximum allowed participants    |
| <code>status</code>           | ENUM(upcoming, scheduled, ongoing, completed, cancelled) | Lifecycle status                |
| <code>announcement</code>     | TEXT, NULLABLE                                           | Public announcement text        |
| <code>created_at</code>       | DATETIME                                                 | When the tournament was created |

**Primary Key:** `tournament_id` **Foreign Keys:** `game_id` → `games.game_id` **Relationships:**

| Related Entity          | Type      | Description                                     |
|-------------------------|-----------|-------------------------------------------------|
| games                   | Many-to-1 | Many tournaments can feature one game           |
| tournament_participants | 1-to-Many | One tournament has many registered participants |

**Cardinality:** - 1 game → 0..N tournaments - 1 tournament → 0..N participants (up to `max_participants`) **Rules & Constraints:** - **Minimum lead time:** Tournament `start_date` must be at least 7 days from the creation date - Registration is only allowed when `status = scheduled` (open for registration) - Cannot exceed `max_participants`; capacity is re-verified at payment confirmation to prevent race conditions - Status transitions: `upcoming` → `scheduled` → `ongoing` → `completed`; cannot revert from `completed` - `entry_fee` must be set before registration opens - Only staff/admin can create or update tournaments

## ENTITY 12 — tournament\_participants

**Description:** The junction (bridge) entity connecting users to tournaments. Each record represents one registration — a user joining a specific tournament. Supports both online self-registration (with GCash proof upload) and walk-in registrations added by staff. **Attributes:**

| Attribute         | Type                                | Description                                            |
|-------------------|-------------------------------------|--------------------------------------------------------|
| participant_id    | INT, PK, AUTO_INCREMENT             | Unique participant record ID                           |
| tournament_id     | INT, FK → tournaments.tournament_id | The tournament being joined                            |
| user_id           | INT, FK → users.user_id             | The registering customer (0 for walk-in)               |
| registered_by     | INT, FK → users.user_id, NULLABLE   | Staff who added a walk-in (NULL for self-registration) |
| ign               | VARCHAR(100), NULLABLE              | In-game name / alias                                   |
| contact_number    | VARCHAR(20), NULLABLE               | Contact number at registration                         |
| gcash_proof       | VARCHAR(255), NULLABLE              | Uploaded GCash receipt filename                        |
| payment_status    | ENUM(pending, paid)                 | Whether entry fee was confirmed                        |
| placement         | INT, NULLABLE                       | Final finishing position                               |
| prize_amount      | DECIMAL(10,2), NULLABLE             | Prize won (if any)                                     |
| registration_date | DATETIME                            | When registration was submitted                        |

**Primary Key:** participant\_id **Foreign Keys:** - tournament\_id → tournaments.tournament\_id (CASCADE DELETE) - user\_id → users.user\_id - registered\_by → users.user\_id (SET NULL on delete) **Relationships:**

| Related Entity | Type      | Description                                                     |
|----------------|-----------|-----------------------------------------------------------------|
| tournaments    | Many-to-1 | Many participants belong to one tournament                      |
| users          | Many-to-1 | One user can join many tournaments (many participants per user) |

**Cardinality:** - 1 tournament → 0..N participants - 1 user → 0..N tournament registrations - This is effectively a **Many-to-Many** relationship between users and tournaments, resolved through this junction table **Rules & Constraints:** - One registration per registered user per tournament (unique constraint enforced at application level; user\_id = 0 for walk-ins bypasses this) - gcash\_proof upload is required for self-registered participants paying online - Registration slot is only confirmed ( payment\_status = paid ) after GCash payment is verified via PayMongo - **Capacity re-check:** max\_participants on the tournament is verified again at payment confirmation to prevent overbooking - Walk-in participants added by staff have registered\_by set to the staff user\_id ; user\_id = 0 - placement and prize\_amount are populated by admin after the tournament concludes

## RELATIONSHIP SUMMARY TABLE

| From                | Cardinality | To                            | Foreign Key                              |
|---------------------|-------------|-------------------------------|------------------------------------------|
| users               | 1 → N       | notifications                 | notifications.user_id                    |
| users               | 1 → N       | reservations                  | reservations.user_id                     |
| users               | 1 → N       | gaming_sessions               | gaming_sessions.user_id                  |
| users               | 1 → N       | reservation_cancellations     | reservation_cancellations.user_id        |
| users               | 1 → N       | reschedules                   | reschedules.rescheduled_by               |
| users               | 1 → N       | transactions                  | transactions.user_id                     |
| users               | 1 → N       | tournament_participants       | tournament_participants.user_id          |
| consoles            | 1 → N       | reservations                  | reservations.console_id                  |
| consoles            | 1 → N       | gaming_sessions               | gaming_sessions.console_id               |
| reservations        | 1 → 0..1    | reservation_cancellations     | reservation_cancellations.reservation_id |
| reservations        | 1 → 0..1    | reschedules                   | reschedules.reservation_id               |
| gaming_sessions     | 1 → N       | session_extensions            | session_extensions.session_id            |
| gaming_sessions     | 1 → N       | transactions                  | transactions.session_id                  |
| games               | 1 → N       | tournaments                   | tournaments.game_id                      |
| tournaments         | 1 → N       | tournament_participants       | tournament_participants.tournament_id    |
| users ↔ tournaments | M → N       | (via tournament_participants) | —                                        |

\*Total: 12 entities | 15 direct relationships | 1 many-to-many resolved via junction table\*